

author1hash=654d899beffa7085e946e3fab1c1049afamily=Alireza Ghaderi, fa-
milyi=A. G., given=Asadullah Dal, giveni=A. D. author1hash=7e32656830b5b690307e71cacc59db53fa
Lamé, familyi=A. L., given=et al., giveni=e. a., suffix=Shuvam Keshari, suffi-
xi=S. K. author1hash=776828efa2737f017d05c533df1f8041family=Canva,
familyi=C. author1hash=175994b725e57b60418287475d6f172cfamily=Dr. Frank
Antwerpes, familyi=D. F. A., given=et al., giveni=e. a., suffix=Bijan Fink, suffi-
xi=B. F. author1hash=d8a033d987772b6ce88da226c95b9363family=Incorporate,
familyi=I., given=Eraser Labs, giveni=E. L. author1hash=2d2b0d998977e6379aad61f1b56ac3c1family=
familyi=M. author1hash=a82b421177a01bad3815de6fbe90a483family=SA.,
familyi=S., given=Eyeware Tech, giveni=E. T. author1hash=a82b421177a01bad3815de6fbe90a483famil
familyi=S., given=Eyeware Tech, giveni=E. T. author1hash=a82b421177a01bad3815de6fbe90a483famil
familyi=S., given=Eyeware Tech, giveni=E. T. author1hash=1cef24d7254bc83735c8681a5e47ce64family
familyi=T., given=Carla, giveni=C. author1hash=1cef24d7254bc83735c8681a5e47ce64family=Team,
familyi=T., given=Carla, giveni=C. author1hash=1cef24d7254bc83735c8681a5e47ce64family=Team,
familyi=T., given=Carla, giveni=C.

Implementierung eines Eye-Tracking Systems in den Fahrsimulator der Hochschule Karlsruhe

Projektarbeit

Jonas Verleger, Lenard Haas

... April 2025

Inhaltsverzeichnis

1	Einführung	4
1.1	Zielsetzung	4
1.2	Lösungsansatz	4
2	Eye Tracker Grundlagen	5
2.1	Allgemeines	5
2.2	Marktanalyse	5
2.3	Beam Eye Tracker	6
3	Implementierung Eye Tracker	7
3.1	Einrichtung im Fahrsimualtor	7
3.2	Beam Eye Tracker SDK	10
3.3	Schnittstelle zu Carla	10
4	Carla Umgebung	13
4.1	Aufbau Testszenario in Carla	13
4.2	Sensor Auswertung	14
4.2.1	Verschiedene Sensortypen	14
4.2.2	Instance Segmentation Sensor	16
4.2.3	Optimierung der Performance	16
4.3	Darstellung der Daten in Carla	17
5	Korrektur gekrümmte Leinwand	20
5.1	Problemursache	20
5.1.1	Hauptproblem	20
5.1.2	Variabilität des Problems	21
5.1.3	Problem der Sitzposition	21
5.2	Lösungsansatz	22
5.2.1	Mathematischer Ansatz e-Funktion	24
5.2.2	Wieso gibt es mit diesem Lösungsansatz jetzt ein Limit	26
5.2.3	Quadratische-Funktion	27
5.2.4	V-Funktion mit abgeflachtem Beginn und Ende	30
6	Fazit	33
6.1	Auswertung und Zusammenfassung	33
6.2	Ausblick	33

1 Einführung

Die Analyse des menschlichen Fahrverhaltens erfordert bedingt durch die Komplexität der Eingangsvariablen einen großen Betrachtungswinkel. Hierbei hilft es Simulationen zu nutzen, um alle erdenklichen Szenarien ohne Gefahren testen zu können. Das Institut für Energieeffiziente Mobilität (IEEM) entwickelt hierzu ein Simulationsframework mit Fahrsimulatoren in unterschiedlicher Realitätsnähe.

Als Softwarebasis dient das Open-Source-Projekt CARLA. In CARLA ist es möglich, verschiedene Verkehrsszenarien und Wetterbedingungen einzustellen. Die Server-Client Struktur von CARLA lässt ebenso mehrere Probanden in demselben Verkehrsszenario zu.

Für die Analyse des Fahrverhaltens wird mindestens die Auswertung der Fahrzeugdaten (z. B.: der zurückgelegte Weg, einwirkende Kräfte) benötigt. Da das menschliche Fahrverhalten höchst komplex ist, eignen sich für eine ausreichende Analyse ebenfalls die Fahrerentscheidungen. Nach einem Fahrversuch können Probanden befragt werden, warum sie welche Entscheidungen getroffen haben. Dennoch bietet es sich an Echtzeitdaten der Probanden aufzuzeichnen.

Eine große Rolle zum Treffen von Entscheidungen spielt die Sicht. Hierdurch wird klar, was die Probanden bei der Testfahrt sehen, worauf diese sich konzentrieren und welchen Fahrweg sie nehmen werden. Für die Datenerhebung der Sicht eignet sich ein sogenannter Eye-Tracker.

1.1 Zielsetzung

Im Rahmen dieser Projektarbeit soll ein kamerabasierter Eye-Tracking Algorithmus entwickelt werden, der in die CARLA Simulationsumgebung integriert wird.

Dazu werden mögliche Eye-Tracker auf die Anwendbarkeit und Integration in CARLA analysiert. Ebenfalls wird eine Simulation in CARLA erstellt, um den Eye-Tracker zu testen. Weitere Vorgaben sind die Projektion des Blickpunktes auf die Leinwand und eine Ausgabe der aufgezeichneten Daten.

1.2 Lösungsansatz

Um das Erreichen der Ziele gewährleisten zu können, wird sich zuerst mit den Grundlagen von Eye-Trackern und der CARLA-Simulationsumgebung befasst. Für die Integration in das Gesamtsystem wird ein objektorientierter Python Code entwickelt. Der Code soll den Blickpunkt des Probanden als Pixel ausgeben und mit CARLA austauschen können. Ebenfalls wird die benötigte Rechenleistung minimiert. Die Ausgabe der erhobenen Daten erfolgt in Echtzeit über ein Fenster und wird im .csv-Format abgespeichert.

2 Eye Tracker Grundlagen

Die Aufzeichnung der Blickposition im Raum kann mithilfe einer Kamera und einer Positionsrechnung durchgeführt werden. Ein Algorithmus zur Berechnung lässt sich über die Lichteigenschaften herleiten. Lichtstrahlen verlaufen direkt von einer Quelle zu einem Objekt und werden mit dem gleichen Austrittswinkel reflektiert, wie der Eintrittswinkel. Die Brechung von Licht durch Linsen oder Gase wird bei dieser Annahme ignoriert. Über die Kenntnis, wo sich die Kamera, welche die Augen aufzeichnet, befindet, wird es durch eine anfängliche Kalibrierung mit fest definierten Punkten ermöglicht, Rückschlüsse auf die Blickposition zu berechnen. So entsteht ein Eye-Tracking Algorithmus. **QUELLE**

Im Sinne des zeitlichen Rahmens des Projektes wird eine Marktanalyse durchgeführt, um verfügbare Eye-Tracker auf deren Anwendbarkeit, Integration in CARLA und Kosten zu untersuchen. Einen eigenen Algorithmus zur Berechnung der Blickposition und der Integration in ein funktionierendes Programm zu entwickeln, kann selbst eine Projektarbeit widerspiegeln.

2.1 Marktanalyse

Ein erster Softwarekandidat ist ein Open Source Tracking Projekt auf GitHub **QUELLE**. Die Ausgabe dieses Trackers ist in Abbildung 1 zu erkennen. Das System orientiert sich hauptsächlich an den Augen und der Nase der Testperson. Dadurch kann es recht genau bestimmen, in welche Richtung der Kopf einer betrachteten Person geneigt ist. Zudem kann die Software das Blinzeln der Augen erkennen. Hierdurch kann theoretisch eine Analyse durchgeführt werden, welche die Reaktionszeit mit einem Blinzeln in Relation setzt. In diesem Projekt fällt ein solcher Betrachtungswinkel weg.

Die Software schätzt nicht die Blickposition der Testperson auf dem Bildschirm, sondern gibt nur an, in welcher Lage sich der Kopf befindet. Diese Daten könnten aber als Grundlage dienen für eine Weiterentwicklung. Da allerdings nur die Ausrichtung des Gesichts und nicht der Pupillen in die Ausgabe einfließt, erwarten wir keine besonders genauen Ergebnisse durch diese Herangehensweise.

Eine weitere Option stellt das open source Projekt eyegaze Tracker dar. Diese Software erfasst nicht die Gesichtsausrichtung, sondern die Pupillen. Dadurch kann sie Aussagen über die grobe Blickrichtung treffen. So wie in Abbildung 2 zu erkennen, kann bspw. die Aussage getroffen werden, dass die Testperson nach links schaut. Auch hier wird keine Blickposition auf dem Bildschirm geschätzt. Allerdings werden die Pupillen erfasst und die grobe Blickrichtung bestimmt. Die Blickposition kann gegebenenfalls mit den vorhandenen Daten durch eine Weiterentwicklung des vorhandenen open source Codes bestimmt werden. Dies würde allerdings sehr viel Entwicklungsaufwand bedeuten.

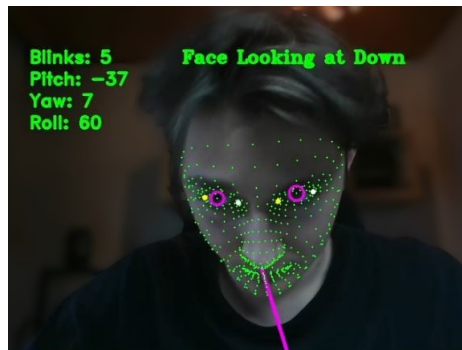


Abbildung 1: Ausgabe des Face-Gaze-Trackers, Quelle: `[lit'code'github'facegazetracker]`

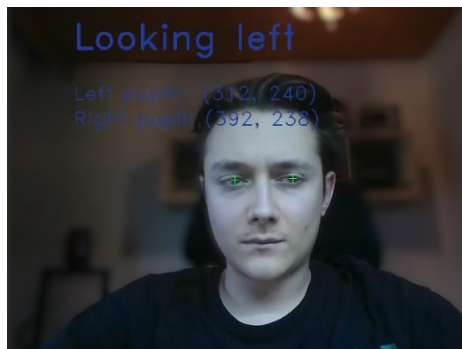


Abbildung 2: Ausgabe des Eyegaze-Trackers, Quelle: `[lit'code'github'gazetracking]`

Neben den oben genannten Projekten wurden auch noch einige weitere auf ihre Einsetzbarkeit in diesem Projekt untersucht. Diese hatten größtenteils ähnliche Eigenschaften und Einschränkungen. Es wurde sich nach viel Recherche und Experimenten für den Beam Eye Tracker von Eyeware Tech entschieden. Dieser hat unsere Anforderungen grundlegend erfüllt und soll als Basis für die weitere Entwicklung dienen.

	Vorteile	Nachteile
Eyeware Tech SA Beam Eye Tracker	...	...
Tobii Pro Spark	...	...
SR Research EyeLink 1000 Plus	...	...
Pupil Labs Core	...	...

2.2 Beam Eye Tracker

Genauere Beschreibung von Beam Eye Tracker

Auf der Seite der Beam SDK auf jeden Fall den Abschnitt "API Overview" hier mit reinbringen. Dort steht einiges über die Funktionsweise des Beam Eye Tracker

Hier auch dazu schreiben, dass es preiswert ist und dass wir es geschafft haben für die Uni eine kostenlose Vollversion auf Lebenszeit zu erhalten

3 Implementierung Eye Tracker

3.1 Einrichtung im Fahrsimulator

Eyeware Tech setzt bei ihrem Beam Eye Tracker auf einfache Hardware. Dieser arbeitet mit einer einfachen USB Webcam oder mit einem an den Computer angeschlossenen Smartphone-Kamera. Damit der Einsatz des Eye Trackers im Fahrsimulator gelingt, müssen einige Punkte beachtet werden.

Die Positionierung der Kamera ist sehr wichtig. Diese soll laut Eyeware Tech mittig zum Bildschirm ausgerichtet sein [**lit·software·beam·eyetracker**]. Die Sitzposition der Testperson hingegen muss nicht mittig zur Kamera sein. Die Testperson kann auch relativ zur Kamera weiter links oder weiter rechts sitzen. In den Eye Tracker Einstellungen muss auch die Kameraposition und der Neigungswinkel korrekt eingestellt werden. Die besten Ergebnisse konnten erzielt werden, als die Kamera im Fahrsimulator mittig auf dem Display in der Mitte des Armaturenbretts angebracht war. Die Kameraposition wurde im Eye Tracker als untenügend mit einem Neigungswinkel von 0° angegeben.

Grundlage für gute Ergebnisse ist es die Kalibrierung, welche der Beam Eye Tracker integriert hat, vor jeder Sitzung im Fahrsimulator durchzuführen. Besonders wenn mehrere Personen nacheinander fahren, die sich von ihrer Körpergröße stark unterscheiden. Die in den Beam Eye Tracker integrierte Kalibrierung besteht aus fünf Punkten, die nacheinander auf der Leinwand erscheinen. Vier

Punkte werden dabei in die vier Ecken der Leinwand positioniert. Der fünfte Punkt befindet sich in der Mitte der Leinwand. Während der Kalibrierung muss die Testperson nacheinander auf diese Punkte schauen. Sofern die Testperson einen Punkt fest fokussiert hat, klickt sie mit der Maus auf diesen Punkt. Der Eye Tracker verknüpft dann die aktuelle Blickposition mit den Koordinaten des Punktes auf der Leinwand. Nach diesem Prinzip kalibriert sich der Eye Tracker selbst und verknüpft die Position der Pupillen mit einer Position auf der Leinwand.

Da die Leinwand sehr groß ist und das Kalibrierungsfenster des Eye Trackers nicht verkleinert werden kann, befinden sich die Kalibrierungspunkte teilweise so weit in den Ecken, dass sie vor allem unten nicht mehr auf der Leinwand angezeigt werden. Das macht es schwer sie anzuklicken. Außerdem sollten während der Kalibrierung keine großen Kopfbewegungen gemacht werden. Es hat sich in Experimenten gezeigt, dass das die Tracking Genauigkeit verringern kann. Da die Punkte durch die breite Leinwand aber außerhalb des Sichtfelds der Testperson liegen, muss diese zwangsweise eine Kopfbewegung machen.

Eine Herangehensweise um diese Probleme zu umgehen ist es, die Kalibrierung immer zu zweit durchzuführen. Die Testperson ist dafür verantwortlich auf die Kalibrierungspunkte zu schauen und die zweite Person klickt auf diese mit der Maus. Da die Punkte teilweise außerhalb des Sichtfeldes liegen und die Testperson den Kopf nicht zu sehr bewegen sollte, ist es am besten, wenn diese nicht direkt auf den jeweiligen Punkt schaut, sondern leicht daneben. Wenn der Punkt bspw. unten rechts im Eck erscheint, dann schaut die Testperson bei der Kalibrierung etwas weiter nach links. Zum einen wird dadurch die Kalibrierung insgesamt verbessert, da der Kopf nicht bewegt werden muss. Zusätzlich wird dadurch ein Fehler leicht abgeschwächt, welcher durch die Krümmung der Leinwand entsteht. Zwischenzeitlich wurde im Rahmen dieser Projektarbeit auch der Ansatz einer eigenen Kalibrierung verfolgt. Diese hat vorausgesetzt, dass der Eye Tracker bereits einmal sehr grob mit dem integrierten Kalibrierungsfenster von Beam kalibriert wurde. Bei jedem Start des Programms wurde das selbst erstellte Kalibrierungsfenster geöffnet. Dieses hat nach demselben Prinzip wie das im Beam Eye Tracker integrierte Kalibrierungsfenster gearbeitet. Allerdings war das selbst entwickelte Kalibrierungsfenster in seiner Größe anpassbar. Dadurch wurden die Punkte im Sichtfeld der Testperson angezeigt und nicht an den Rändern der Leinwand. Eine schematische Abbildung des Kalibrierungsfensters ist in Abbildung 3 zu sehen. In der Realität hatte das Fenster einen weißen Hintergrund, um die Lichtverhältnisse relativ zum Beam Kalibrierungsfenster zu verbessern. Die Ergebnisse der Kalibrierung wurden anschließend verwendet, um die Rohdaten des Eye Trackers mit diesen zu transformieren, damit die Kalibrierung Einfluss auf die Schätzwerte der Blickposition nimmt. Die daraus resultierten Ergebnisse wiesen allerdings keine wirkliche Verbesserung auf. Es wird daher empfohlen die integrierte Standard-Kalibrierung des Eye Trackers zu verwenden. Der Code für die selbst entwickelte Kalibrierung ist allerdings weiterhin in auskommentierter Form vorhanden und kann bei Bedarf wieder reaktiviert und weiterentwickelt werden.

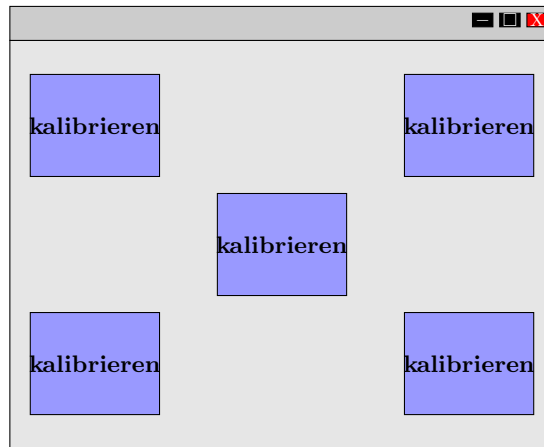


Abbildung 3: Schematische Darstellung des selbst entwickelten Kalibrierungsfensters

Während der Kalibrierung und der gesamten Verwendung des Eye Tracking Systems darf nur die Testperson im Fahrsimulator sitzen. Eine zweite Person würde das Tracking System beeinflussen, da nicht selektiert werden kann, welche Augen getrackt werden sollen und welche nicht. In Zukunft muss hier eine Art Scheuklappe eingebaut werden, um dieses Problem zu lösen.

Ein weiterer wichtiger Faktor ist die Beleuchtung im Fahrsimulator. Das Kalibrierungsfenster des Beam Eye Trackers hat einen schwarzen Hintergrund. Wenn keine externe Lichtquelle verwendet wird, dann ist es definitiv zu dunkel, als dass eine ordentliche Kalibrierung stattfinden könnte. Selbst wenn während der Kalibrierung gute Lichtverhältnisse herrschen, reicht die Helligkeit der Leinwand während des Fahrens oft nicht aus, um verlässliche Ergebnisse zu liefern. Es konnte eine starke Verbesserung der Trackingqualität erzielt werden, nachdem im Fahrsimulator eine externe Lichtquelle während der gesamten Kalibrierung und der gesamten Fahrzeit eingesetzt wurde. Neben externen Mitteln, um die Lichtverhältnisse zu verbessern, können auch in der Windows Kamera App Anpassungen vorgenommen werden. Dazu muss in den Einstellungen der Kamera App unter "Kameraeinstellungen" die Option "Erweiterte Steuerelemente für Fotos und Videos" aktiviert werden. Dadurch wird ein Helligkeitsregler verfügbar, der die Helligkeit des Kamerabildes systemweit verändert. Je nach aktuellen Lichtverhältnissen kann ein Erhöhen oder Verringern der Bildhelligkeit den Kontrast der Pupillen relativ zur Umgebung verbessern. Bei der Wahl der Kamera sollte eine Kamera verwendet werden, welche eine Auflösung von mindestens 1920x1080 besitzt.

In Abbildung 4 sind die Kameraposition und die externe Lichtquelle aufgebaut im Fahrsimulator zu erkennen. Licht und Kamera sollten zu einem späteren



Abbildung 4: Positionierung von Kamera und externer Lichtquelle, Quelle: [lit'software'canva]

Zeitpunkt noch besser in das System integriert werden. Zu Testzwecken hat der Aufbau allerdings gute Ergebnisse geliefert.

3.2 Beam Eye Tracker SDK

Eyewear Tech bietet beim Kauf des Beam Eye Tracker das Beam-Software Development Kit (SDK) an. Das SDK ermöglicht es die Ausgabe des Eye Trackers in selbst geschriebenen Anwendungen zu verwenden [lit'website'beam'sdk'documentation]. Auf der Website des Beam SDK wird in Python geschriebener Beispielcode zur Verfügung gestellt. In diesem Code werden die nötigen Schritte für eine Verbindung mit der API des Eye Trackers durchgeführt. Der Code wurde auch als Grundlage für die Implementierung des Eye Trackers in das von uns geschriebene Programm verwendet [lit'code'beam'sdk].

Die Funktionsweise der API ist vereinfacht in Abbildung 5 zu erkennen. Der Tracker Server ist die Bezeichnung für den Eye Tracker selbst. Der Client ist das selbst geschriebene Programm, welches die Trackerdaten über die API entgegennimmt. Die beiden Hauptinformation, welche vom Server an den Client übergeben werden sind die Daten zur Kopfposition (head pose data) und die Daten zur Blickposition (screen gaze data). Die Kopfposition gibt Auskunft über die Translation und Rotation des Kopfes der Testperson. Die Blickposition gibt Auskunft auf welchen Punkt auf dem Bildschirm die Testperson schaut. Diese beiden Systeme laufen unabhängig voneinander und dienen unterschiedlichen Einsatzzwecken. Das Kopftracking kann bspw. dazu verwendet werden, um die Kamera in einer Simulationswelt zu bewegen. Das könnte Sinn machen, wenn keine große Leinwand wie im Fahrsimulator vorhanden ist. Durch einen Blick nach links, könnte sich auch die Kamera im Fahrsimulator mit nach links bewege-

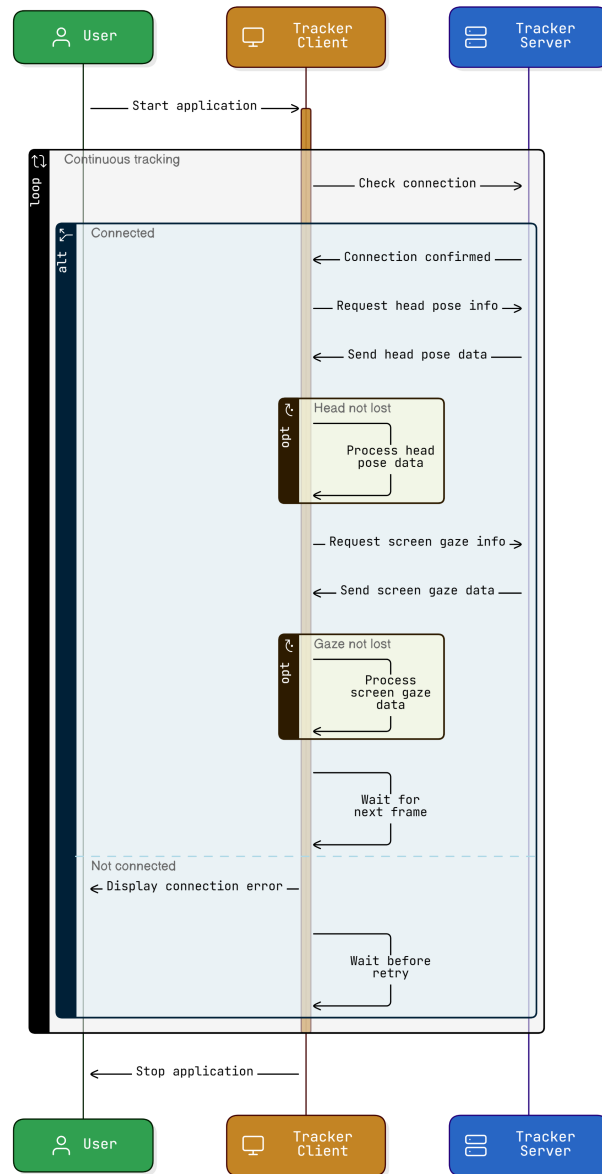


Abbildung 5: Visuelle Darstellung des Codes zur Beam SDK, Quelle: [lit'code'beam'sdk], [lit'software'eraserio]

gen. Für den Einsatzzweck im Fahrsimulator ist diese Funktion durch die große Leinwand allerdings nicht notwendig. Die Daten werden dennoch übermittelt,

falls in Zukunft doch ein Anwendungsfall entstehen sollte. Die Gaze Daten sind das eigentlich Relevante. Diese geben eine Schätzung ab, auf welchen Punkt die Testperson auf der Leinwand schaut. Sofern eine Testperson vom Eye Tracker erkannt ist, werden in kontinuierlichen Abständen diese Daten vom Eye Tracker an den Client übermittelt. Zusätzlich dazu werden auch noch andere Daten wie bspw. die confidence übermittelt. Dabei handelt es sich um einen Wert, welcher mit jeder geschätzten Blickposition mit übermittelt wird. Dieser gibt an, wie sicher sich der Eye Tracker mit seiner Schätzung der aktuellen Blickposition ist.

3.3 Schnittstelle zu Carla

Der Eye Tracker und Carla arbeiten größtenteils getrennt voneinander. Die visuelle Ausgabe des Eye Trackers erfolgt durch einen kleinen schwarzen Punkt, welcher die aktuell, geschätzte Blickposition auf der Leinwand anzeigt. Dieser schwarze Punkt befindet sich in einem transparenten Fenster, welches über die RGB-Sensorausgabe gelegt ist. Es gibt auch einen sogenannten Textmanager. In diesem werden für den Nutzer unter anderem die grobe Blickrichtung in Textform angezeigt. Zum Beispiel "schaut nach oben links". Eine weitere Übergabe der Eye Tracker Daten erfolgt in eine Funktion, welche eine Krümmungs-Korrektur durchführt. Da die Leinwand im Fahrsimulator nicht gerade ist, sondern halbkreisförmig aufgebaut ist, muss diese Funktion den daraus resultierenden Fehler eliminieren. Das Problem der Krümmung und die dafür entwickelte Korrektur werden in einem späteren Abschnitt detailliert erläutert.

Die einzige direkte Schnittstelle zwischen Carla und den Daten des Eye Trackers ist die Übergabe der Eye Tracker Daten an die Auswertung des Instance Segmentation Sensors. Diese Schnittstelle funktioniert so, wie in Abbildung 6 dargestellt. Der Carla Server sendet die Sensordaten des Instance Segmentation Sensors an den Carla Client. Diese kommen bereits um den Faktor zehn kleiner an als die Sensordaten des RGB-Sensors. Der Carla Client übergibt die Sensordaten an die Auswertung. Bei der Auswertung werden die Rohdaten des Eye Trackers, welche bereits eine Krümmungs-Korrektur erfahren haben, dann mit den Sensordaten des Instance Segmentation Sensors verarbeitet. Es erfolgt die Ausgabe, in welcher die Gruppenzugehörigkeit und die ID des Objektes enthalten sind. Diese Ausgabe wird dann im sogenannten "Text Manager" in Form von Text auf dem Bildschirm dargestellt. Die Darstellung des schwarzen Punktes bedient sich ebenfalls an den korrigierten Eye Tracker Daten, hat aber nichts mit dem Text Manager oder Carla zu tun.

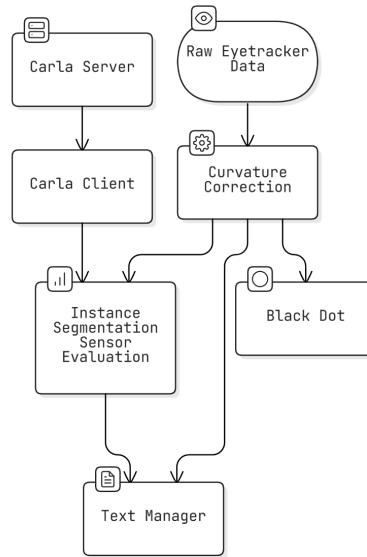


Abbildung 6: Graphische Darstellung der Verarbeitung der Raw-Daten des Eye Trackers, Quelle: [lit'website'carla'documentation]

4 Carla Umgebung

4.1 Aufbau Testszenario in Carla

Um den Eye Tracker in Zusammenhang mit Carla testen zu können, muss eine Testumgebung in Carla angelegt werden. In Abbildung 7 sind die groben Schritte zur Erzeugung einer Testumgebung zu erkennen. Hier ist das Zusammenspiel zwischen dem Carla Server und dem Carla Client besonders hervorgehoben. Der Host-PC ist in diesem Fall der Computer, auf welchem der Fahrsimulator ausgeführt wird. Auf diesem ist Windows als Betriebssystem installiert. Im gesamten Zeitraum der Projektarbeit wird der Carla Server immer lokal auf diesem Computer laufen. Der Carla Server ist für die eigentliche Simulation sowie der Aktoren in ihr verantwortlich. Der Client läuft auf demselben Computer und kann Anfragen an den Server senden. Auf diese Weise kann der Client die Simulation aktiv beeinflussen oder Daten wie bspw. Positionsdaten oder Bilddaten vom Server anfragen. [lit'website'carla'documentation]

Der Carla Server kann heruntergeladen werden und direkt als ausführbares Programm gestartet werden. Anders als der Carla Client, welcher selbst in Python geschrieben werden muss. Dafür kann dieser auch nach den eigenen Anforderungen individuell aufgesetzt werden. In der Dokumentation von Carla gibt

es einige Beispiele wie ein Client aufgebaut werden kann, nach welchen sich der Client für diese Projektarbeit auch orientiert [`lit'code'api`]. Zur Serververbindung nutzt er das Application Programming Interface (API) des Carla Servers. Der Client wurde so geschrieben, dass er zuerst versucht, eine Verbindung zum Carla Server aufzubauen. Anschließend holt er sich wichtige Informationen über die simulierte Welt und mögliche Spawnpunkte für bspw. Fahrzeuge vom Server. Unter Actors werden in diesem Kontext unter anderem Fahrzeuge und Sensoren bezeichnet. Der Client fordert eine Liste aller in der aktuell simulierten Welt vorhandenen Fahrzeuge und Sensoren an und „erstört“ diese anschließend. Somit wird sichergestellt, dass keine Aktoren aus einer vorherigen Sitzung in die neue Sitzung mitgebracht werden. Nachdem die simulierte Welt wieder einen fest definierten Ausgangszustand angenommen hat, beginnt der Client damit, einen Mercedes Sprinter zu spawnen. Abbildung 8 stellt die Fortsetzung des Prozesses dar. Nachdem das Fahrzeug vom Server in der Welt erzeugt wurde, fordert der Client die Erzeugung von zwei Sensoren an. Beide Sensoren sollen an das erzeugte Fahrzeug geknüpft sein. Der erste Sensor soll den Rot-Grün-Blau (RGB)Farbraum erfassen. Der zweite Sensor soll die Instance Segmentation erfassen. Auf diese und die Verarbeitung der daraus resultierenden Daten wird weiter unten genauer eingegangen.

4.2 Sensor Auswertung

4.2.1 Verschiedene Sensortypen

Der RGB-Sensor liefert ein Bild an den Client, welches einer Kameraaufnahme gleicht. Durch die roten, grünen und blauen Farbanteile kann ein Bild erzeugt werden, welches für die Testperson im Fahrsimulator die simulierte Welt in Farbe darstellt. Die simulierte Welt kann durch diesen RGB-Sensor wie durch eine Kameralinse betrachtet werden. Der Server berechnet die Welt in Zahlen und der Sensor ermöglicht es der Testperson im Fahrsimulator diese auf zahlen basierende Welt visuell wahrzunehmen. Die Objekte und die Farben der Welt werden so dargestellt, wie sie für den Menschen natürlich sind. Ein Baum hat die Form eines Baums und besitzt grüne Blätter und einen braunen Stamm. In Carla kann bei Bedarf auch ein Lidar-Sensor platziert werden. Dieser erzeugt ebenfalls ein Bild der simulierten Umgebung. Dieses würde den Baum dann aber als eine Art Punktwolke in einer schwarzen Umgebung darstellen. Ein Lidar-Sensor würde die Welt in einer anderen Art und Weise wahrnehmen, wodurch auch ein anderes Bild im Vergleich zu einem RGB-Sensor entstehen würde.

4.2.2 Instance Segmentation Sensor

Der Instance Segmentation Sensor ist in seiner Funktion etwas spezieller. Dieser kategorisiert die Objekte in der Welt in Gruppen ein und weist ihnen IDs zu. Er färbt gewisse Objektgruppen nach ihrer jeweiligen Gruppenzugehörigkeit.

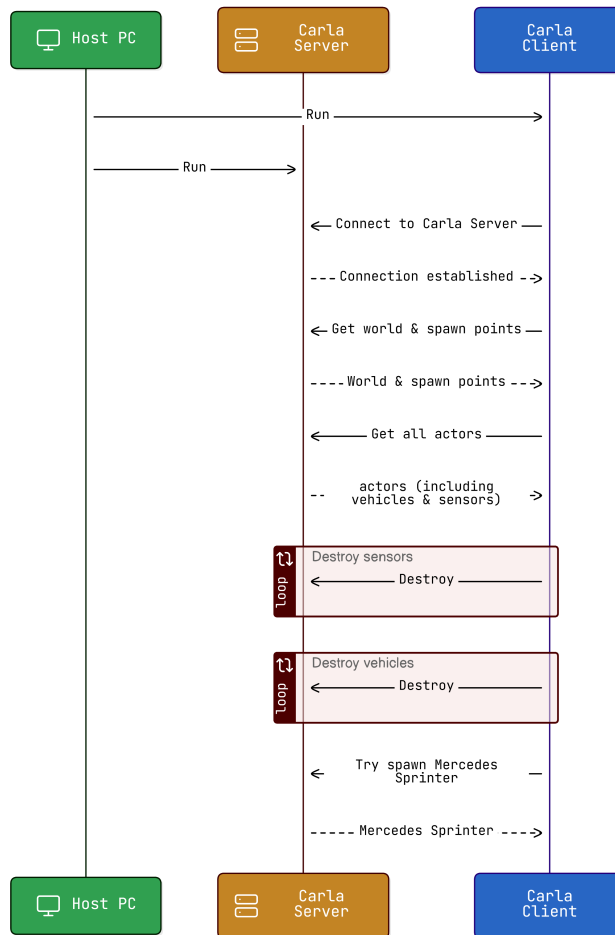


Abbildung 7: Visuelle Darstellung des Codes zur Carla Testumgebung Teil 1/2, Quelle: [lit'software'eraserio],[lit'website'carla'documentation]

Bspw. werden Straßen hellgrün und Gebäude dunkelblau gezeichnet. Zusätzlich besitzt jedes Objekt eine eindeutige Identifikationsnummer (ID). So hat jedes Fahrzeug in der simulierten Welt eine eindeutige, ihm zugewiesene ID. Damit sowohl die Gruppenzugehörigkeit, als auch die ID in die Bildinformation der Sensorausgabe übergeben werden können, gibt es zwei Codierungsmechanismen. Bei der Sensorausgabe handelt es sich wie beim RGB-Sensor um eine Farbbildausgabe bestehend aus rot-, grün- und blau-Anteilen. Die Zugehörigkeit eines Objekts in eine bestimmte Gruppe wird durch den Rot-Anteil des Bildes an der Stelle des Objektes bestimmt. Die Tabelle 1 stellt die korrekte Zuweisung zwischen dem Rot-Wert und der jeweiligen Eingruppierung dar. Im Code wur-

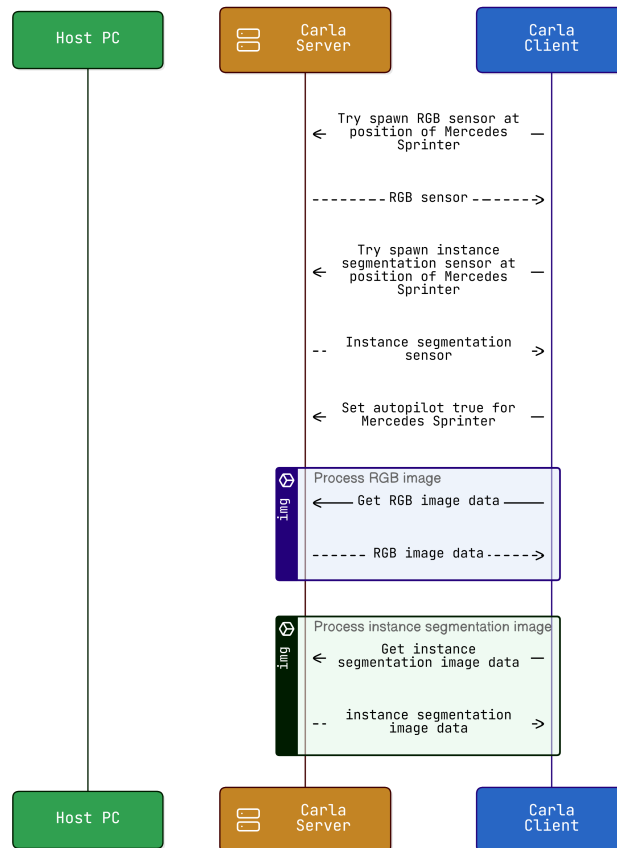


Abbildung 8: Visuelle Darstellung des Codes zur Carla Testumgebung Teil 2/2, Quelle: [lit'software'eraserio],[lit'website'carla'documentation]

de sich auf diese Zuweisung gestützt. Es muss berücksichtigt werden, dass bei unterschiedlichen Versionen von Carla eine unterschiedliche Zuweisung erfolgt [lit'website'carla'documentation].

Die ID wird in den grün- und blau-Anteil des Bildes codiert. Liegt bspw. ein grün-Anteil von 22 und ein blau-Anteil von 232 vor, dann lautet die ID des Objektes "22-232".

Der Client fordert die Sensorausgabe des Instance Segmentation Sensors in regelmäßigen Abständen vom Server an. Anschließend extrahiert er die RGB-Werte und wertet diese nach den oben beschriebenen Zuordnungen aus. Die Auswertung erfolgt dabei immer genau an einem Pixel. Die Position des Pixels ist definiert durch die aktuelle Ausgabe der geschätzten Blickposition vom Eye Tracker.

Rot-Wert	Gruppe
0	Unlabeled
1	Roads
2	SideWalks
3	Building
4	Wall
5	Fence
6	Pole
7	TrafficLight
8	TrafficSign
9	Vegetation
10	Terrain
11	Sky
12	Pedestrian
13	Rider
14	Car
15	Truck
16	Bus
17	Train
18	Motorcycle
19	Bicycle
20	Static
21	Dynamic
22	Other
23	Water
24	RoadLine
25	Ground
26	Bridge
27	RailTrack
28	GuardRail

Tabelle 1: Zuweisung Rot-Wert und Objektgruppe bei Carla Version 0.9.15, Quelle: [lit'website'carla'documentation]

4.2.3 Optimierung der Performance

Der Rechenaufwand, den der Server leisten muss, um die Sensorausgabe zu generieren, ist nicht zu vernachlässigen. Bei der RGB-Ausgabe sind nicht viele Abstriche möglich, da die Testperson ein Bild von hoher Auflösung für die große Leinwand im Fahrsimulator benötigt. Die Sensorausgabe des Instance Segmentation Sensors kann hingegen auch mit einer geringeren Auflösung generiert werden. Dadurch wird Rechenleistung gespart und da die Sensorausgabe nur im Hintergrund benötigt wird, macht es für die Testperson auch keinen bemerkbaren Unterschied.

Damit das Zusammenspiel zwischen Eye Tracker und Sensor allerdings wei-

terhin funktioniert, müssen die Werte des Eye Trackers ebenfalls auf die herunterskalierte Sensorausgabe umgerechnet werden. In Abbildung 9 ist diese Skalierung grafisch dargestellt. Durch den Faktor zehn werden aus den Originalbilddaten jeweils 10x10 Pixel auf ein einzelnes Pixel herunterskaliert. Alle Pixel innerhalb dieses 10x10 Rasters, wie bspw. das Pixel bei Position $x=7$ und $y=4$ werden dann auf ein Pixel bezogen. Dadurch wird offensichtlich die Genauigkeit verringert, da der Eye Tracker zwar Tracking Daten bezogen auf die Originalauflösung ausgibt, dann aber auf die herunterskalierte Instance Segmentation Sensorausgabe beziehen muss. Bei einer Reduzierung der Auflösung um Faktor zehn wurden allerdings keine bemerkbaren, negativen Einflüsse auf die Funktion des Eye Trackers festgestellt.

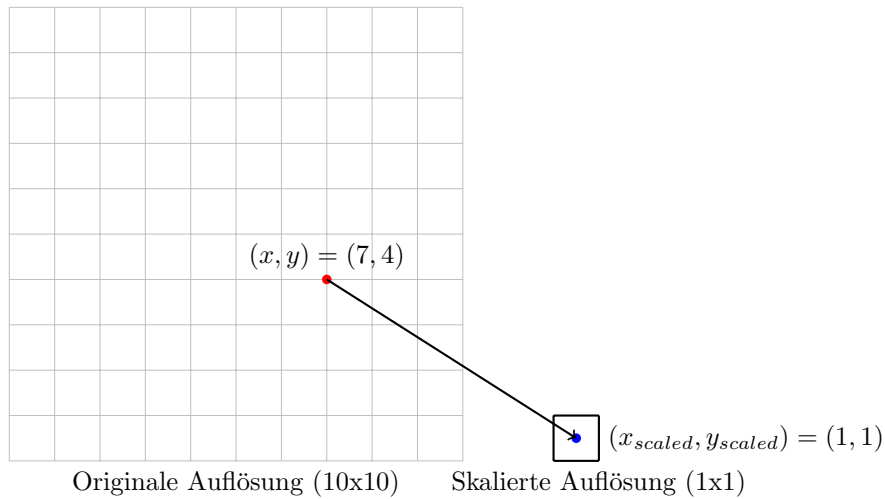


Abbildung 9: Herunterskalierung der Eye-Tracking-Daten um Faktor 10

4.3 Darstellung der Daten in Carla

Die Daten des Eye Trackers werden nicht direkt in Carla dargestellt, sondern in einem transparenten Windows Fenster, welches über die Carla Client Anwendung geschoben wird. Die Inhalte dieses transparenten Fensters können allerdings in Zukunft bei Bedarf in die Carla Anwendung selbst implementiert werden.

In Abbildung 10 ist die graphische Darstellung der Daten des Eye Trackers im transparenten Fenster zu erkennen. Das erste Anzeigeelement ist der lila gefärbte Kreis. Dieser zeigt die geschätzte Blickposition des Eye Trackers an. Bei den Werten handelt es sich um die Rohdaten, welche noch nicht durch unsere Korrekturfunktion bearbeitet wurden. Als zweites wird hier der rote Punkt angezeigt. In der Realität ist das der Mauszeiger. Zur besseren Sichtbarkeit

wurde dieser durch einen roten Kreis hervorgehoben. Der Mauszeiger wurde dazu verwendet, um die Performance des Eye Trackers messbar zu machen. Die Testperson hat den Mauszeiger bewegt und auf diesen geschaut. Dadurch war der Wert der tatsächlichen Blickposition bekannt und konnte mit dem Wert für die geschätzte Blickposition verglichen werden. Während der normalen Verwendung des Eye Trackers spielt dieser aber keine Rolle. Als drittes Element wird der schwarze Punkt angezeigt. Dieser liefert genauso wie der lila farbene Kreis die geschätzte Blickposition auf der Leinwand. Allerdings werden für die Darstellung des schwarzen Punktes die von uns korrigierten Werte verwendet. Der schwarze Punkt ist dementsprechend der eigentlich geltende Wert. Das vierte Anzeigeelement ist die textbasierte Ausgabe. In dieser werden verschiedene Werte ausgegeben, welche gleich noch detaillierter beschrieben werden. Das kleine türkisfarbene Fenster oben links ist die Ausgabe des Instance Segmentation Sensors. Diese kann bei der Verwendung des Eye Trackers im Hintergrund laufen und muss nicht weiter beachtet werden.

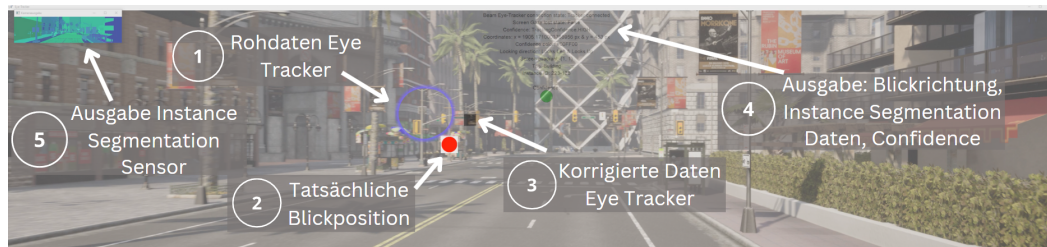


Abbildung 10: Darstellung der Daten in Carla, Quelle: [lit'software'carla], [lit'software'canva]

In Abbildung 11 sind die textbasierten Daten aus Anzeigeelement vier in Abbildung 10 nochmals genauer dargestellt. Es werden Informationen über den Verbindungsstatus des Eye Trackers, die aktuell geschätzten Blickkoordinaten, die grobe Blickrichtung und die Instance Segmentation Auswertung gegeben. Die 'Looking Direction' gibt dabei die grobe Blickrichtung der Testperson an. Der 'Tag' ist die Einordnung des Objektes auf welches die Testperson schaut in eine Objektgruppe und die 'Instance ID' die eindeutige Identifikationsnummer für selbiges Objekt. Der grüne Punkt unten gibt die aktuelle Tracking Confidence des Eye Trackers nochmals in Form einer Ampel aus. Dadurch kann sofort erkannt werden, wenn der Eye Tracker die Testperson nicht mehr korrekt erfasst.

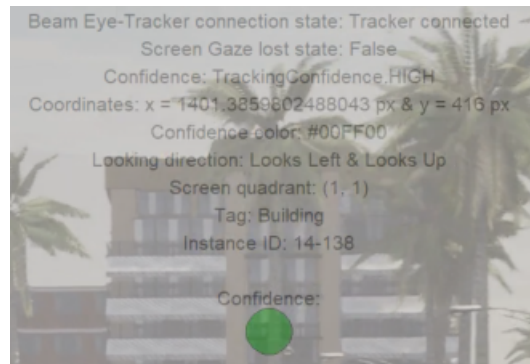


Abbildung 11: Darstellung textbasierten Daten in Carla, Quelle: [lit'software'carla], [lit'software'canva]

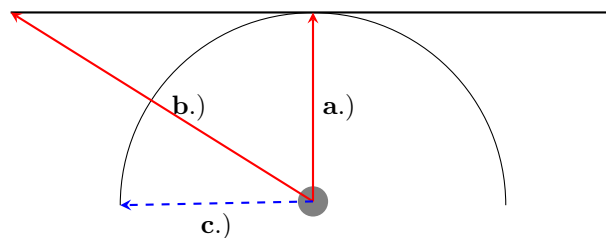


Abbildung 12: Visualisierung der verzerrenden Wirkung einer gekrümmten Leinwand auf die Ergebnisse des Eye Trackers

5 Korrektur gekrümmte Leinwand

5.1 Problemursache

5.1.1 Hauptproblem

Die Leinwand des Fahrsimulators ist nicht flach sondern besitzt eine halbkreisförmige Form. Der Eye Tracker hat keine Einstellmöglichkeit, um eine gekrümmte Leinwand zu berücksichtigen. Dadurch entstehen starke Abweichungen zwischen der vom Eye Tracker geschätzten Blickposition und der realen Blickposition. Abbildung 12 visualisiert dieses Problem.

In Abbildung 12 ist die gekrümmte Leinwand, sowie eine flache Leinwand zu erkennen. Aus Sicht des Eye Trackers existiert nur die flache Leinwand. Ein kleiner grauer Punkt visualisiert die Position der Testperson im Fahrsimulator. Pfeil a.) zeigt direkt von der Testposition in einem Winkel von 0° auf die Mitte der Leinwand. Es ist zu erkennen, dass es in diesem Punkt keine Abweichung

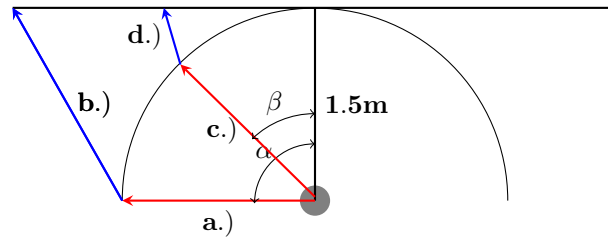


Abbildung 13: Visualisierung der nichtlinearen Transformation

zwischen der flachen Leinwand und der gekrümmten Leinwand gibt. Aus diesem Grund ist zu erwarten, dass der Eye Tracker in diesem Punkt die genauesten Ergebnisse liefert.

Pfeil b.) zeigt ein zweites Szenario. Die Testperson schaut in diesem Fall nach links. Es ist zu erkennen, dass sie mit ihrem Blick die gekrümmte Leinwand auf der linken Seite etwas links von der Mitte trifft. Wenn der Blick der Testperson allerdings imaginär verlängert wird und dadurch auf die imaginäre, flache Leinwand trifft, wird die Abweichung erkennbar. Es ist zu erwarten, dass der Eye Tracker die Blickposition auf den linken Bildschirmrand schätzt, obwohl die reale Blickposition einen gewissen Abstand zum Bildschirmrand besitzt. In einem optimierten System würde der Eye Tracker die Blickposition, welche mit Pfeil b.) dargestellt ist, nur dann schätzen, wenn die Testperson auf den Punkt schaut, welcher mit Pfeil c.) dargestellt ist.

5.1.2 Variabilität des Problems

Es besteht kein gleichbleibender Zusammenhang zwischen der Blickposition auf der flachen Leinwand und der gekrümmten Leinwand. In Abbildung 13 sind zwei Beispiele zu erkennen. In Beispiel 1 wird der Blick auf die gekrümmte Leinwand mit dem Pfeil a.) dargestellt. Die Transformation auf die flache Leinwand wird mit Pfeil b.) visualisiert. Bei diesem Beispiel schaut die Testperson in einem Winkel $\alpha = 90^\circ$ nach links. Zwischen dem Blick a.) und dem transformierten Blick b.) stellt sich ein Winkel ein. In Beispiel 2 wird nicht an den Bildschirmrand der gekrümmten Leinwand geschaut, sondern in die Mitte. Der Winkel $\beta = 45^\circ$ stellt sich ein. Auch zwischen dem Blick c.) und dem transformierten Blick d.) stellt sich ein Winkel ein. Aus der Abbildung ist zu abzulesen, dass der Winkel zwischen b.) und a.) kleiner ist als der Winkel zwischen c.) und d.). Das zeigt, dass die Transformation zwischen der gekrümmten und der flachen Leinwand nicht konstant ist. Das Problem ist an den Rändern stärker ausgeprägt als in der Nähe der Mitte.

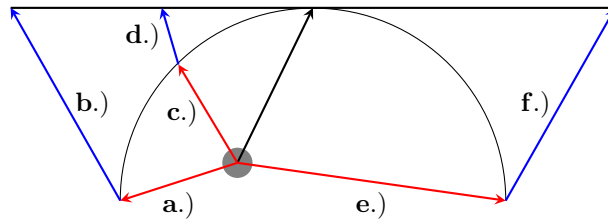


Abbildung 14: Visualisierung der Verschiebung der Sitzposition der Testperson

5.1.3 Problem der Sitzposition

In den Abbildungen 12 und 13 wurde die Sitzposition der Testperson im Fahrsimulator immer im Mittelpunkt der gekrümmten Leinwand angekommen. In der Realität sitzt die Testperson nicht in der Mitte des Halbkreises, sondern näher in Richtung Leinwand. Ein Mensch hat ein Sichtfeld von ca. 200° [lit'website'doccheck'fov]. Würde die Testperson in der Mitte des Halbkreises sitzen, dann würde die Leinwand 180° des Sichtfeldes umfassen. Da die Testperson aber näher an der Leinwand sitzt und somit nicht mehr in der Mitte des Halbkreises, befinden sich die Ränder der Leinwand weiter hinten und das Sichtfeld wird noch weiter ausgereizt. Kopfbewegungen im Eye Tracker sind nicht optimal. Da die Ränder der Leinwand nur durch eine kleine Kopfbewegung in das Sichtfeld rutschen, resultiert daraus eine verringerte Genauigkeit des Eye Trackers in der Nähe der Bildschirmränder.

Eine weitere Abweichung von den Annahmen weiter oben ist die um links verschobene Sitzposition der Testperson im Fahrsimulator. Bisher wurde angenommen, dass die Testperson direkt gegenüber der Mitte der Leinwand sitzt. In der Realität ist sie durch das nachgebaute Fahrzeugcockpit leicht nach links versetzt. Dadurch verhält sich die Transformation von flacher auf gekrümmte Leinwand links anders als rechts. Abbildung 14 visualisiert diese Einflüsse.

Es ist zu erkennen, dass in diesem Szenario vor allem zwischen der Blickrichtung c.) und der Transformation d.) kaum Fehler besteht. Durch die Sitzposition weiter links wird dort eine geringere Abweichung zwischen gekrümmter und flacher Leinwand bestehen. Experimente am Fahrsimulator haben diese Annahme bestätigt. Besonders in der Mitte der linken Seite hat der Eye Tracker gut geschätzt. Weiter links und sehr weit rechts wurde die Genauigkeit ohne eine Transformation auf die gekrümmte Leinwand wesentlich schlechter.

5.2 Lösungsansatz

Wie im Abschnitt davor bereits beschrieben, muss eine Transformation zwischen den beiden Leinwänden durchgeführt werden. Die Richtung der Transformation wird dabei durch die Datengrundlage bestimmt. Die Daten stammen vom

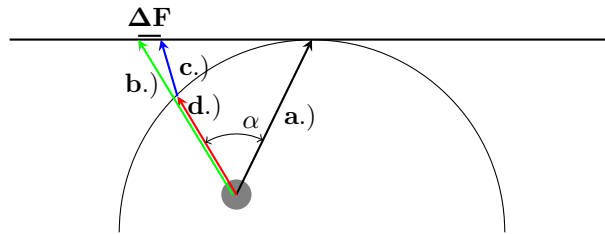


Abbildung 15: Graphische Darstellung des Lösungsansatzes für die Transformation

Eye Tracker und dieser geht von einer flachen Leinwand aus. Dementsprechend muss die Transformation von der flachen Leinwand auf die gekrümmte Leinwand stattfinden. Die Sitzposition und der Blickwinkel bestimmen, wie die Transformation durchzuführen ist.

Bei der Transformation ist nur die Horizontale von Relevanz. Vertikale Pixel müssen nicht transformiert werden, weil die Leinwand zwar horizontal gekrümmt ist aber nicht vertikal. Es besteht kein vertikaler Fehler zwischen der flachen und der gekrümmten Leinwand. Zu Zwecken der Nachvollziehbarkeit werden zu Beginn zwei einzelne Pixel betrachtet, welche von der flachen Leinwand auf die gekrümmte Leinwand projiziert werden sollen. Beide Pixel sollen sich auf der linken Hälfte der flachen Leinwand befinden. Abbildung 15 stellt den resultierenden Fehler bei gegebener Sitzposition und Blickrichtung dar.

In Abbildung 15 ist der Vektor von der Sitzposition zur Mitte der gekrümmten Leinwand durch a.) gegeben. Die Testperson schaut bezogen auf die gekrümmte Leinwand in Richtung Pfeil d.). Der Eye Tracker würde jetzt allerdings den Punkt auf welchen Pfeil b.) zeigt erkennen. Dieser würde wiederum mit einem anderen Punkt auf dem Halbkreis korrespondieren. Welcher in der Abbildung nicht dargestellt wird, weil es sonst zu unübersichtlich wird. Wichtig ist, dass die Ausgabe des Eye Trackers so transformiert wird, dass er anstatt den Punkt auf welchen Pfeil b.) zeigt, den Punkt, auf welchen Pfeil c.) zeigt, ausgibt. Es muss also eine Transformation um den Fehler ΔF stattfinden.

Die Berechnung von ΔF ist ein mathematisches Problem. Durch die nach links verschobene Sitzposition und die fehlenden Längenangaben der einzelnen Pfeile ist die Berechnung recht komplex. Ein alternativer Ansatz wäre eine Funktion, welche grob eine Transformation durchführt und vor Ort feinjustiert wird. In Abbildung 16 ist zu erkennen, dass bei einem Blick in die Mitte der linken Hälfte der gekrümmten Leinwand, eine leichte Korrektur nach rechts notwendig ist. Schaut man auf den Pfeil c.) aus Abbildung 16 wird klar, dass auf der rechten Seite der Halbkugel eine Korrektur nach links notwendig ist und diese stärker ausfallen muss, als auf der linken Seite. Aus den vorherigen Abbildungen ist zudem herauszulesen, dass das Problem umso stärker wird, je näher sich der Blick den Rändern der Leinwand annähert. Dabei ist es egal, ob linke Hälfte

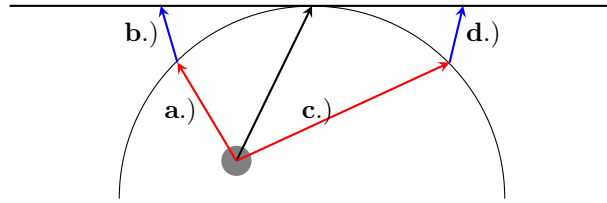


Abbildung 16: Bestimmung der notwendigen Korrektur auf beiden Halbsseiten des Halbkreises

oder rechte Hälfte. Allerdings befindet sich der Punkt, welcher keine Korrektur benötigt, nach wie vor in der Mitte der Leinwand, unabhängig von der Sitzposition der Testperson.

5.2.1 Mathematischer Ansatz e-Funktion

Um die Transformation durchzuführen wird folgende selbst entwickelte Formel verwendet:

$$k = e^{c \cdot (w-p)} \quad (1)$$

Dabei ist:

k - Korrektur-Wert

c - Korrektur-Koeffizient (zum Einstellen der Korrektur)

w - Bildschirmbreite in Pixel geteilt durch zwei

p - Zu korrigierender Pixel

Des weiteren gelten folgende Formeln:

$$p_{k,l} = p + k \quad (2)$$

Dabei ist:

$p_{k,l}$ - Korrigierter Pixel links

p - Zu korrigierender Pixel

k - Korrektur-Wert

Die Gleichung refeq:2 wird nur auf der linken Bildschirmhälfte angewandt. Der Korrektur-Wert k bestimmt dabei die Pixel, um welche man die Position des Eye-Trackers verschieben muss. Da es sich um die linke Bildschirmhälfte handelt, muss nach rechts korrigiert werden. Auf den vom Eye-Tracker bestimmten Pixel muss also der Korrektur-Wert aufsummiert werden. Wir müssen sozusagen vom linken Bildschirmrand weg eine Korrektur in Richtung Bildschirmmitte durchführen.

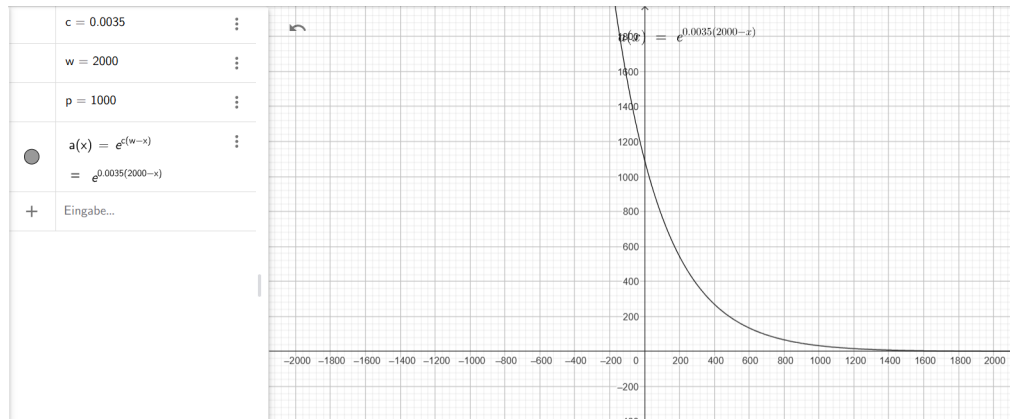


Abbildung 17: Sinnvolle Werte für den Korrekturfaktor linke Bildschirmhälfte

In der Abbildung ist zu erkennen für welchen Korrektur-Koeffizienten bei einer Leinwand mit 4000 Pixeln Breite sinnvolle Werte zu erzielen sind. Bei 4000 Pixeln Gesamtbreite, also 2000 Pixeln auf der linken Bildschirmhälfte wäre $c=0.0035$ ein realistischer Wert.

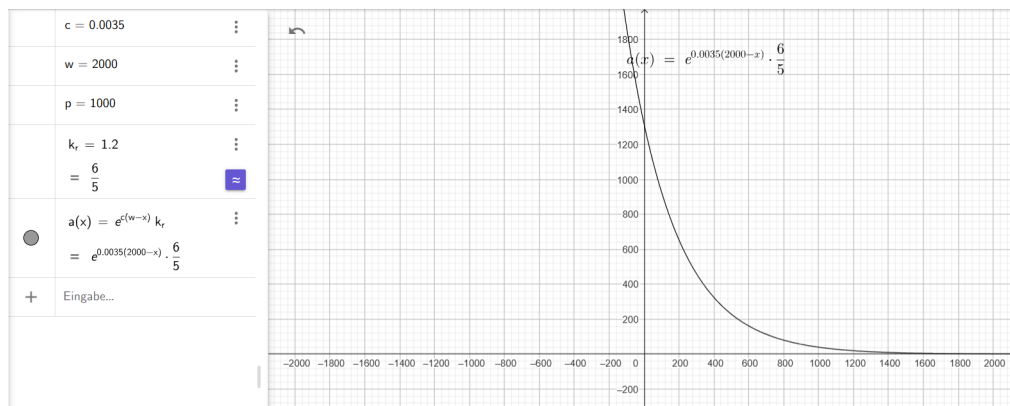


Abbildung 18: Sinnvolle Werte für den Korrekturfaktor rechte Bildschirmhälfte

In der Abbildung sind sinnvolle Werte für die rechte Bildschirmhälfte zu erkennen. Beim Korrekturfaktor c ändert sich nichts. Der zusätzlich hinzugekommene Korrekturfaktor $p_{k,r}$ wurde in diesem Beispiel mit 1.2 ausgewählt.

Die Feinabstimmung muss experimentell am Fahrsimulator durchgeführt werden.

Der Korrekturfaktor muss für die rechte Bildschirmhälfte leicht verändert gerechnet werden. Angenommen die halbe Bildschirmbreite w beträgt 2000 Pixel. Die größte Korrektur findet am Rand statt also bei $2*w=4000$ Pixel. Dort ist es dann: $2000-(4000-4000)=2000$. In der Mitte soll keine Korrektur stattfinden. Dort ist es dann: $2000-(4000-2000)=0$.

$$k = e^{c*(w-(2*w-p))} \quad (3)$$

$$p_{k,r} = p - k * k_r \quad (4)$$

Dabei ist:

$p_{k,r}$ - Korrigierter Pixel links

p - Zu korrigierender Pixel

k - Korrektur-Wert

k_r - Zusätzlicher Korrektur-Koeffizient für rechte Seite

Die Gleichung refeq:3 wird nur auf der rechten Bildschirmhälfte angewandt. Die bei der rechten Seite wird anders als bei der linken Seite nach links und nicht nach rechts korrigiert. Aus diesem Grund wird in der Formel mit einem negativen Vorzeichen gerechnet. Zusätzlich gibt es einen weiteren Koeffizient k_r . Dieser stellt sicher, dass die Korrektur auf der rechten Bildschirmseite leicht stärker ausfällt, da die durch die links ausgerichtete Sitzposition der Testperson notwendig ist.

Der mathematische Ansatz wurde im Fahrsimulator getestet und es konnten zwar Verbesserungen festgestellt werden, allerdings gab es noch einige Probleme und Optimierungsbedarf. Die Korrektur in der Mitte des Bildschirms ist wesentlich zu schwach. Auch im mittleren Bereich ist sie zu schwach. Zudem sind in Richtung Ränder große Einbußen des Sichtfeldes zu bemängeln. Eine detaillierte Auswertung der Ergebnisse wird hier übersprungen, da schon durch bloßes Hinsehen zu erkennen ist, dass diese Funktion ungeeignet ist. 1

5.2.2 Wieso gibt es mit diesem Lösungsansatz jetzt ein Limit

Schauen wir die linke Bildschirmhälfte an. Ab einem gewissen Punkt erreichen wir den $x=0$ auf der flachen Leinwand. Das entspricht in etwa dem letzten drittel

auf der linken Bildschirmhälfte. Ist dieser Punkt erreicht, gibt der Raw-Output des Eye Trackers einen Wert von $x=0$ aus, weil wir auf einer flachen Leinwand den linken Rand erreicht hätten. Gehen wir jetzt auf der gekrümmten Leinwand mit dem Blick noch weiter nach links, dann ändert sich an dem Wert $x=0$ nichts mehr, weil der Eye-Tracker denkt wir wären bereits am Rand. Noch weiter nach links versteht er nicht. Der Korrekturwert erreicht am Rand sein Maximum. Angenommen dieses sei bei Korrektur=700 Pixel. Der linke Rand ist auf der flachen Leinwand bei $x=0$ Pixel. Auf der gekrümmten Leinwand ist dieser Wert allerdings weiter rechts. Ca. beim letzten drittel der linken Bildschirmhälfte. Die linke Bildschirmhälfte umfasst 2222 Pixel. $1/3 \cdot 2222 = 740$ Pixel. Wenn dieser Punkt erreicht ist, dann wird auf diese 740 Pixel der Korrekturfaktor von 700 Pixel addiert. Anschließend ist man bei $740 + 700$ Pixel und somit bei 1140 Pixel. Schaut man weiter nach links wird das vom Eye Tracker nicht erfasst, weil wir bei der flachen Leinwand immernoch bei $x=0$ sind. Auf der gekrümmten Leinwand bleiben wir bei $x=740$ Pixel hängen. Gleichzeitig bleibt bei $x=740$ Pixel der Korrekturwert von 700 Pixel identisch, weswegen eine statische Grenze von 1140 Pixel vom linken Rand entfernt entsteht. Wird der Koeffizient c erhöht, dann wird die Korrekturkurve dadurch steiler. Dadurch wird beim Nullpunkt $x=0$ auf der flachen Leinwand und ca. $x=740$ Pixel auf der gekrümmten Leinwand jetzt ein stärkerer Korrekturfaktor angewandt. Sagen wir der aggressivere Korrekturfaktor ist jetzt nicht mehr 700 Pixel, sondern 900 Pixel. Die neue statische Grenze liegt jetzt bei $740 \text{ Pixel} + 900 \text{ Pixel} = 1640 \text{ Pixel}$ vom linken Rand entfernt. Das bedeutet, dass ein höherer Korrekturfaktor c die Korrektur aggressiver macht und gleichzeitig den verwendbaren Bereich des Eye-Trackers einschränkt. Auf der rechten Bildschirmhälfte tritt der Effekt noch stärker auf, da dort noch zusätzlich mit dem Korrekturkoeffizient k_r gerechnet wird.

Hieraus kann das Fazit gezogen werden: Höherer Korrekturkoeffizient c bedeutet eine aggressivere Korrektur und möglicherweise bessere Ergebnisse. Gleichzeitig wird aber der Sichtbereich, in welchem der Eye Tracker eingesetzt werden kann, verschmälert.

5.2.3 Quadratische-Funktion

Anstatt e-Funktion wird eine quadratische Funktion eingesetzt. Das k_r ist wie bei der e-Funktion nur auf der rechten Bildschirmhälfte anzuwenden.

$$k = c * (w - p)^2 * k_r \quad (5)$$

Die Korrektur mit der neuen Funktion wurde im Labor experimentell getestet. Dabei wurde auch Fine-Tuning durchgeführt, um die Funktion so gut es geht an die realen Gegebenheiten anzupassen. Die besten Ergebnisse konnten durch $c=0.012$ und $k_r = 1.5$ erreicht werden.

Wie in Abbildung 19 zu erkennen ist, besteht jetzt im Vergleich zur e-Funktion eine stärkere Korrektur in der Nähe der Mitte der Leinwand. An den

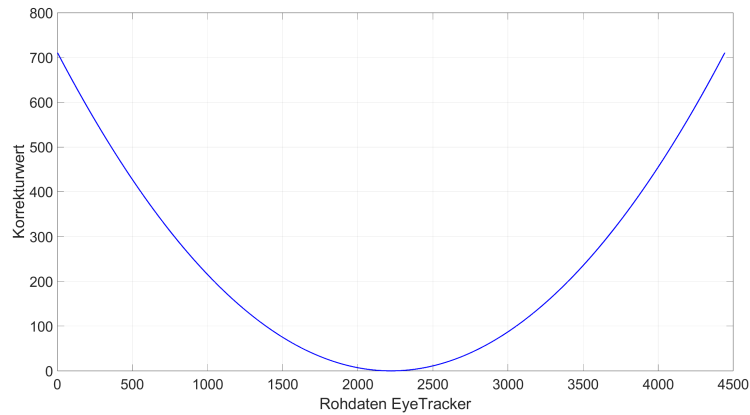


Abbildung 19: Visualisierung quadratische Korrekturfunktion, Quelle: [lit'software'matlab]

Rändern ist der Korrekturwert nicht so hoch wie bei der e-Funktion, wodurch der nutzbare Sichtbereich weniger stark eingeschränkt wird.

Im Fahrsimulator war eine leichte Verbesserung spürbar und es wurde eine Auswertung der Ergebnisse durchgeführt.

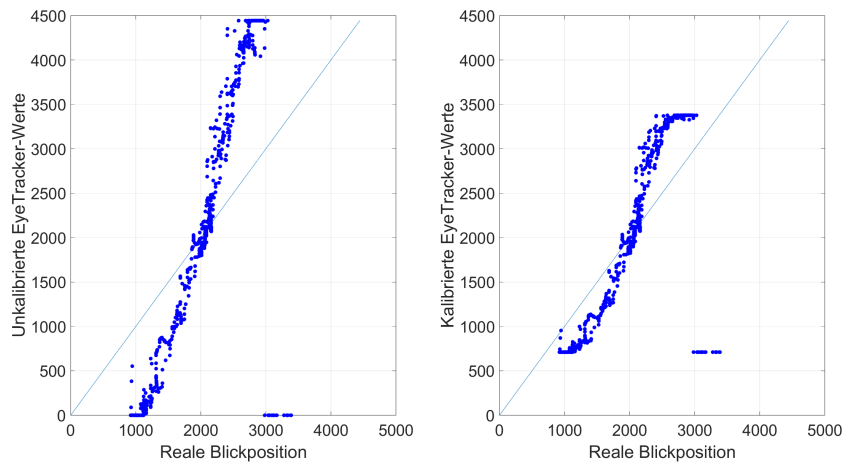


Abbildung 20: Gegenüberstellung reale Blickposition und Eye Tracker-Werte: unkalibriert (links), kalibriert (rechts), Quelle: [lit'software'matlab]

In Abbildung 20 sind die Ergebnisse zu erkennen. Auf der x-Achse ist die

reale Blickposition in Pixel aufgetragen. Dabei entspricht 0 Pixel dem linken Bildschirmrand und 4444 Pixel dem rechten Bildschirmrand. Die reale Blickposition wurde mithilfe des Mauszeigers ermittelt. Die Testperson hat den Mauszeiger über den Bildschirm bewegt und ihn mit den Augen verfolgt. Auf der y-Achse ist die vom Eye Tracker geschätzte Blickposition aufgetragen. Diese ist zur Vergleichbarkeit ebenfalls in Pixel angegeben. Für beide Graphen in Abbildung 20 werden auf der x-Achse dieselben Werte verwendet. Auf der y-Achse werden beim linken Graphen die Rohdaten des Eye Trackers aufgetragen. Beim rechten Graphen werden die von unserer quadratischen Funktion kalibrierten Werte angezeigt. Im Idealfall sollten die blauen Punkte deckungsgleich mit der hellblauen monoton steigenden Gerade sein. Um zu prüfen, dass unsere Kalibrierung gut funktioniert, sollten die blauen Punkte im rechten Graphen zum einen so nahe wie möglich an der blauen Gerade anliegen und zum anderen bessere Ergebnisse als im linken Graphen erzielt werden.

Es ist zu erkennen, dass der Sichtbereich auch bei der Verwendung einer quadratischen Korrekturfunktion eingeschränkt wird. Diese Einschränkung konnte allerdings reduziert werden. Bei der e-Funktion war der maximale Korrekturwert auf der linken Bildschirmhälfte $k=1300$. Bei der abgestimmten, quadratischen Korrekturfunktion liegt das Maximum bei $k=700$. Es stehen somit auf der linken Bildschirmhälfte 600 Pixel mehr für das Tracking zur Verfügung. Die Auswertung stimmt mit den theoretischen Überlegungen soweit überein, dass bei ca. 2222 Pixel ein Fehler von null vorhanden ist. Das liegt daran, weil die gekrümmte Leinwand und eine äquivalente, imaginäre, gerade Leinwand in der Bildschirmmitte bei 2222 Pixel identisch sind. Um die Verbesserungen deutlich zu erkennen, müssen bestimmte Bereiche genauer betrachtet werden:

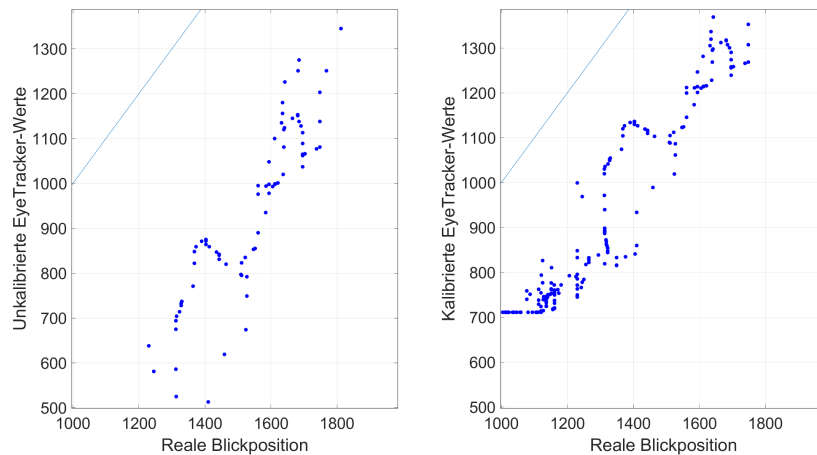


Abbildung 21: Gegenüberstellung reale Blickposition und Eye Tracker-Werte im Detail: unkalibriert (links), kalibriert (rechts), Quelle: [lit'software'matlab]

In Abbildung 21 ist ein Detail-Ausschnitt der Ergebnisse der linken Leinwand-Hälfte zu erkennen. Es ist deutlich zu sehen, dass die Werte nach der Kalibrierung näher an der idealen Geraden liegen. Je näher sich die Blickposition dem linken Bildschirmrand näher, umso geringer wird der absolute Fehler. Es ist allerdings auch festzustellen, dass in Richtung Bildschirmmitte weiterhin zu wenig korrigiert wird. Dort ist generell ein geringer Fehler zwischen gekrümmter und gerade Leinwand zu erwarten. Dennoch ist der dort fast so hoch wie in Randnähe. Würde der Parameter c erhöht werden, dann würde der Fehler reduziert werden. Gleichzeitig würde allerdings auch der nutzbare Erfassungsbereich des Eye Trackers reduziert werden. Es bestehen dieselben Probleme wie bei der e -Funktion, nur weniger stark ausgeprägt.

Eine Funktion 4. Grades scheint auf den ersten Blick sinnvoll. Diese steigt zwar stärker an als eine quadratische Funktion, allerdings besitzt sie in ihrem Ursprung ein Plateau. Dadurch wäre die Korrektur im Bereich der Bildschirmmitte definitiv nicht ausreichend.

5.2.4 V-Funktion mit abgeflachtem Beginn und Ende

Nach den Experimenten im Fahrsimulator wurde aus den daraus gewonnen Erfahrungen eine Funktion gesucht, welche in der Bildschirmmitte einen Korrekturwert von null ausgibt. Die Funktion sollte einen Korrekturfaktor ausgeben, welcher sich relativ zum Abstand zu der Bildschirmmitte erhöht. Dabei sollte dieser Anstieg bereits unmittelbar um die Bildschirmmitte hoch sein, damit der Fehler dort bereits ausreichend verringert wird. An den Rändern soll sie allerdings nicht zu sehr ansteigen, da sonst der nutzbare Sichtbereich reduziert werden würde. Als Kurvendiagramm sollte die Funktion wie ein V aussehen, dessen Tiefpunkt auf der x-Achse bei der Bildschirmmitte, also bei 2222 Pixeln liegt. In Richtung der Ränder sollte die Steigung der Funktion kleiner werden, damit der nutzbare Sichtbereich des Eye Trackers nicht zu sehr eingeschränkt wird. Dieser sollte im Idealfall größer sein, als bei der quadratischen Funktion.

Damit die Steigung stufenlos angepasst werden kann, wurde eine Variable p dafür verwendet. Damit keine negativen Werte herauskommen, muss mithilfe des Betrags sichergestellt werden, dass innerhalb der Basis keine negativen Werte vorkommen. Damit die V-Funktion ihren Tiefpunkt in der Bildschirmmitte hat, muss sie um die Bildschirmmitte nach rechts verschoben werden:

$$k = |x - w|^p \quad (6)$$

Zusätzlich wird der Faktor c eingeführt, welcher ähnlich wie p die Steigung relativ zum Abstand zur Bildschirmmitte beeinflusst. Allerdings nicht ganz so aggressiv wie p . Ein Parameter d wurde eingeführt, um ein Grund-Offset zu erzeugen, welches bei der Feinabstimmung evtl. nützlich sein könnte.

$$k = (c * |(x - w)| + d)^p \quad (7)$$

Damit sich die Ränder der Funktion gegen einen bestimmten Wert annähern, muss eine Dämpfung in die Funktion eingebaut werden. Diese wird durch eine e-Funktion realisiert. Dadurch ist eine schnelle Annäherung an den Grenzwert sichergestellt. Die vollständige Korrekturfaktor-Funktion ist wie folgt definiert:

$$k = (c * |(w - x)| + d)^p * (1 - e^{-g * (|w - x|)}) \quad (8)$$

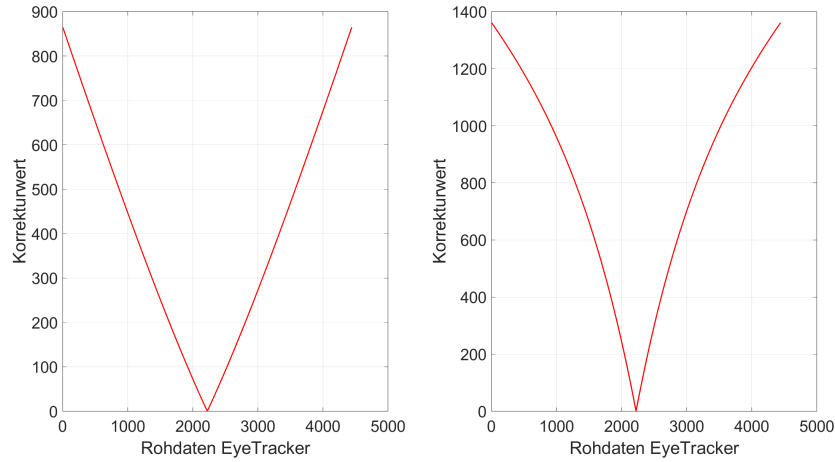


Abbildung 22: Korrekturfunktion: linke Bildschirmhälfte (links), rechte Bildschirmhälfte (rechts), Quelle: `[lit'software'matlab]`

Die graphische Darstellung der Korrekturfunktion ist in Abbildung 22 zu erkennen. Die optimalen Parameter wurden im Fahrsimulator experimentell bestimmt und werden für die Darstellung des Graphen verwendet. Wie in der Abbildung zu erkennen ist, unterschieden sich die Parameter zwischen linker und rechter Leinwandhälfte. Auf der rechten Hälfte muss stärker korrigiert werden. Hier ist die Veränderung der Steigung ab einem gewissen Punkt gut zu erkennen.

In Abbildung 23 sind die Ergebnisse zu erkennen, welche mit der neuen v-förmigen Korrekturfunktion erzielt werden konnten. Es ist eine deutliche Verbesserung gegenüber der quadratischen Korrekturfunktion zu erkennen. Ein eingeschränkter Sichtbereich ist weiter vorhanden. Da ab einem gewissen Punkt allerdings sowieso keine verwertbaren Werte mehr geliefert werden ist das nicht problematisch. Der Eye Tracker korrigiert in der Nähe der Bildschirmmitte jetzt aggressiver und hat damit Erfolg. Auch in Randnähe wieder stärker korrigiert und somit der Fehler reduziert. Durch die unterschiedlichen Parameter für linke und rechte Bildschirmhälfte entsteht im Übergang von linker auf rechte Bildschirmhälfte und anders herum ein kleiner Sprung. Eine weitere Optimierung

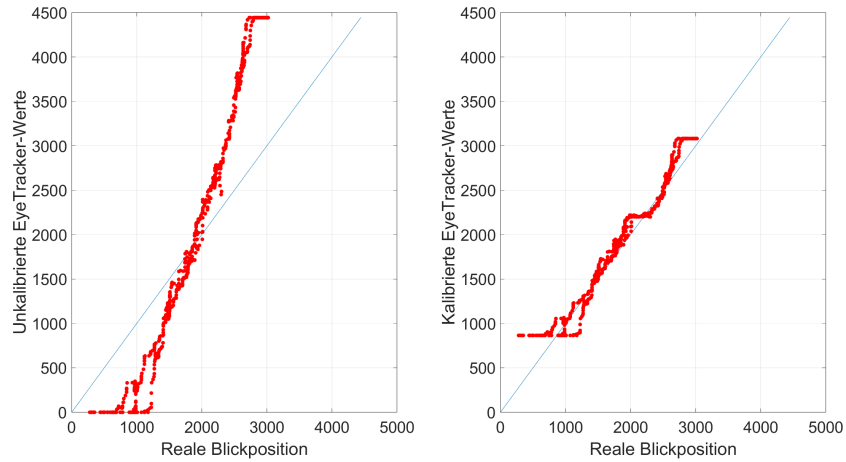


Abbildung 23: Gegenüberstellung reale Blickposition und Eye Tracker-Werte: unkalibriert (links), kalibriert (rechts), Quelle: [lit'software'matlab]

würde den Rahmen dieser Projektarbeit übersteigen.

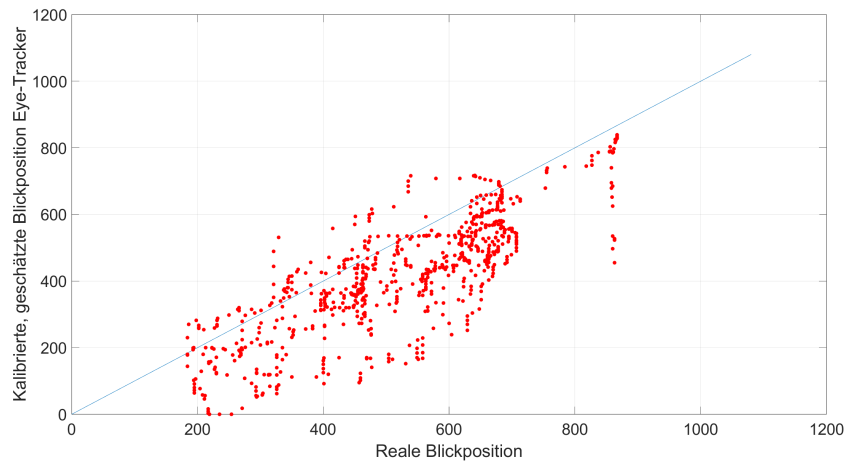


Abbildung 24: Gegenüberstellung reale Blickposition und Eye Tracker-Werte y-Achse kalibriert, Quelle: [lit'software'matlab]

In Abbildung 24 ist die Auswertung der y-Achse nach der Kalibrierung dargestellt. Auf den ersten Blick wirkt es so, als wären die Tracking Ergebnisse auf der y-Achse schlechter als auf der x-Achse. Das liegt allerdings zum Teil

an der anderen Skalierung der y-Achse im Diagramm. Bei den horizontalen Pixelwerten in den Abbildungen weiter oben war die y-Achse von null bis 4444 skaliert. Die y-Achse für die vertikalen Pixelwerte ist allerdings nur von null bis 1080 skaliert. Eine Abweichungen von bspw. 300 Pixeln wirkt im Graphen für die vertikalen Pixel größer als im Graphen für horizontale Pixel. Ein anderer Grund ist, dass keinerlei Optimierung der vertikalen Pixel durch unseren Code vorgenommen wird. Für die Optimierung der horizontalen Pixel durch die Krümmung der Leinwand wurden große Aufwände betrieben. Durch eine Optimierung der vertikalen Pixel könnten evtl. noch weitere Verbesserungen erzielt werden.

Eine abschließende Auswertung der Ergebnisse folgt in einem späteren Abschnitt.

6 Fazit

6.1 Auswertung und Zusammenfassung

Alle Punkte aus der Zielsetzungen konnten im Rahmen der Projektarbeit umgesetzt werden. Die Ergebnisse zeigen, dass es mit einfachen Mitteln wie einer USB-Webcam und sehr preiswerter Software möglich ist, bereits verwertbare Ergebnisse zu generieren. Ursprünglich war angedacht eine Open Source Lösung als Fundament für den Eye Tracker zu verwenden. Nach ausgiebiger Marktrecherche konnte ein passendes Softwarefundament ausfindig gemacht werden. Der Beam Eye Tracker ist zwar nicht Open Source aber die Daten die er durch seine API bereitstellen kann, bieten genug Spielraum, um individuelle Lösungen damit bauen zu können. Eine Verschmelzung zwischen Carla und dem Eye Tracker war ohne größeren Hürden möglich. Die Daten aus dem Instance Segmentation Sensor können zusammen mit den Werten des Eye Trackers ausgewertet werden. Somit ist es möglich nicht nur die geschätzten Koordinaten des Eye Trackers auf der Leinwand auszugeben, sondern auch das betrachtete Objekt innerhalb der Carla Simulationsumgebung zu bestimmen. Während der Implementierung des Eye Tracking Systems sind an manchen Stellen auch Probleme aufgetreten. Die Belichtung war vor allem während der Kalibrierung ein Problem. Durch verschiedene Ansätze und Experimente im Fahrsimulator konnten hier mit externer Belichtung gute Ergebnisse erzielt werden. Ein anderes Problem waren die verfälschten Tracking Daten, verursacht durch die gekrümmte Leinwand. Durch umfassende Analyse des Problems, einige mathematische Ansätze und Experimente im Fahrsimulator, konnte das Problem stark reduziert werden. Die in Abbildung 23 und 24 dargestellten Ergebnisse zeigen, dass gute Ergebnisse erzielt werden konnten. Die Tracking Daten sind allerdings nicht perfekt und lassen sich durch äußere Einflüsse wie schlechte Belichtung, Bedienungsfehler bei der Kalibrierung, Veränderung der Sitzposition nach der Kalibrierung oder zwei Personen gleichzeitig im Fahrsimulator negativ beeinflussen. Der Eye Tracker

könnte allerdings zum aktuellen Entwicklungsstand bereits eingesetzt werden. Anhand der gemessenen Performance kann behauptet werden, dass bereits zum aktuellen Stand nützliche Erkenntnisse während einer Sitzung im Fahrsimulator mit einer Testperson gewonnen werden können. Im Ausblick sollen weitere Schritte und mögliche Ideen präsentiert werden, mit welchen über diesen Low Budget Ansatz hinaus, noch bessere Ergebnisse erzeugt werden könnten.

6.2 Ausblick

- Hier auch darauf eingehen, dass man eine Art Scheuklappe für die Kamera anbauen muss, damit sie den Beifahrer nicht erkennt.
- Fest externe Lichtquelle einbauen
- Kamera besser in das Gesamtsystem integrieren
- In den Anhang der Arbeit das Demo Video einfügen
- Evtl. KI verwenden, um die Tracking Ergebnisse zu optimieren.

Glossar

API Application Programming Interface. 8, 11, 31

ID Identifikationsnummer. 14

RGB Rot, Grün, Blau (Farbraum). 10–14

SDK Software Development Kit. 7

Spawnpunkt Bezeichnet eine 2D oder 3D Koordinate in einer Simulationsumgebung oder in einem Videospiel, an welchem virtuelle Objekte instanziiert werden. 11